

From Bytecode to Native Code in HotSpot JVM

DEV LABS – May 2015

Ionuț Baloșin



From Bytecode to Native Code in HotSpot JVM

About

◆ Ionuț Baloșin

technical leader at [Luxoft.com](https://luxoft.com)

8 years of Java experience

- ◆ 3 years of Java financial applications

author of "*Java Performance And Tuning*" – [Luxoft-Training.com](https://luxoft-training.com) course

From Bytecode to Native Code in HotSpot JVM

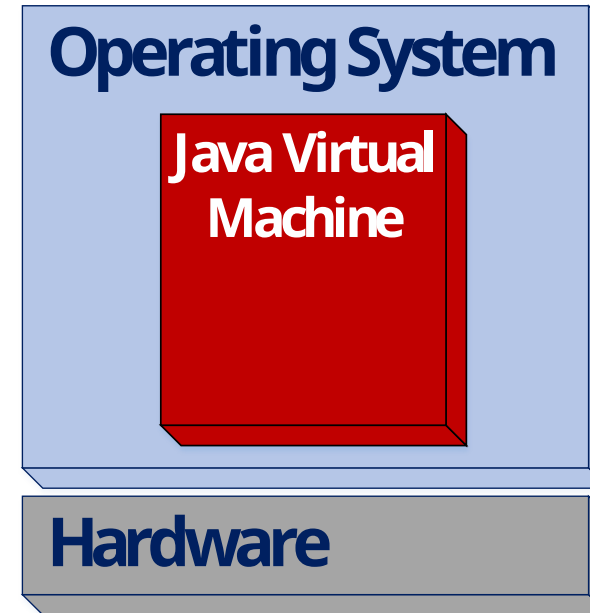
Agenda

- Understanding Bytecode
- How to Disassemble Bytecode
- Bytecode Execution Workflow
 - Interpreter
 - JIT Compiler
- How to see Native Code from Bytecode
- ▣ Case Studies

From Bytecode to Native Code in HotSpot JVM

Java Virtual Machine – some features

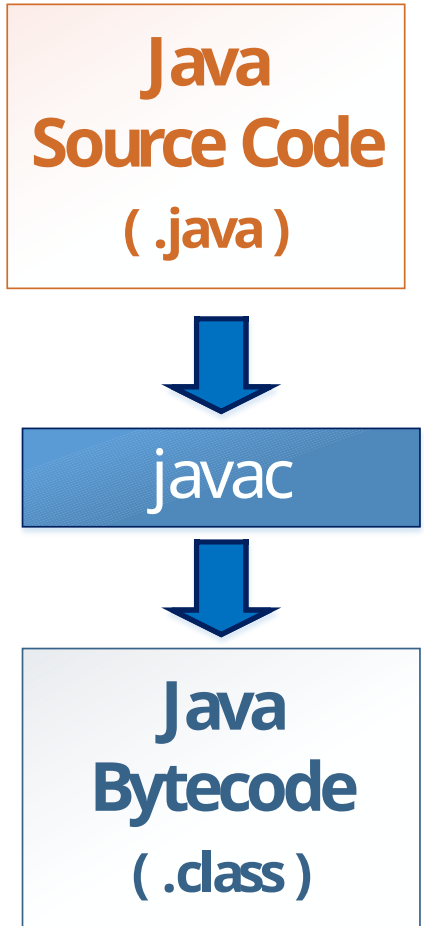
- Process Virtual Machine
- Stack-based
- Symbolic reference



Understanding Bytecode

javac

- ♦ Bytecode is generated using **javac** based on Java Source Code
- ♦ There are about 200 bytecode instructions (**opcodes**) in use
 - each **opcode** is represented by a single byte => hence **bytecode**
 - opcodes** are organized in families
- ♦ Bytecode does not have direct access to any memory location
- ♦ Bytecode does not contain some high-level features present in the Java Source Code
 - E.g. comments, for, while, etc



Understanding Bytecode

javap

- ◆ Disassembled Code is generated using **javap** based on generated Bytecode

- ◆ **javap** - Java Class File Disassembler

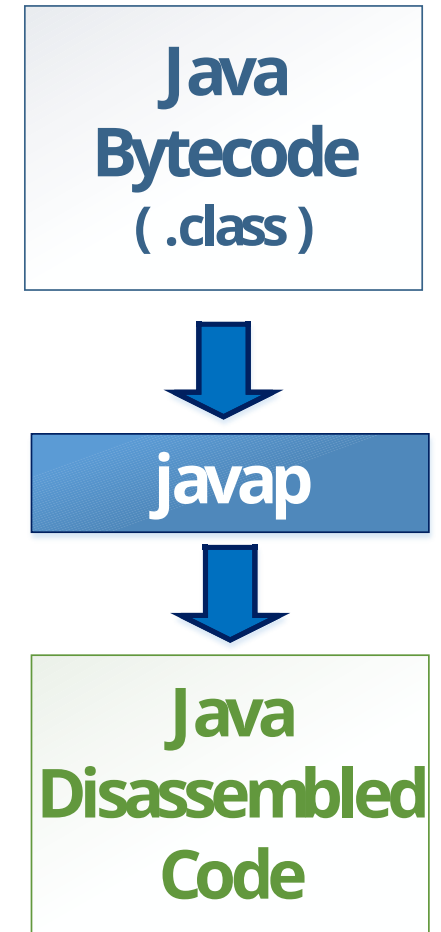
by default shows public and default methods

`javap -c <path_to_class>`

- ◆ IDE Disassemblers:

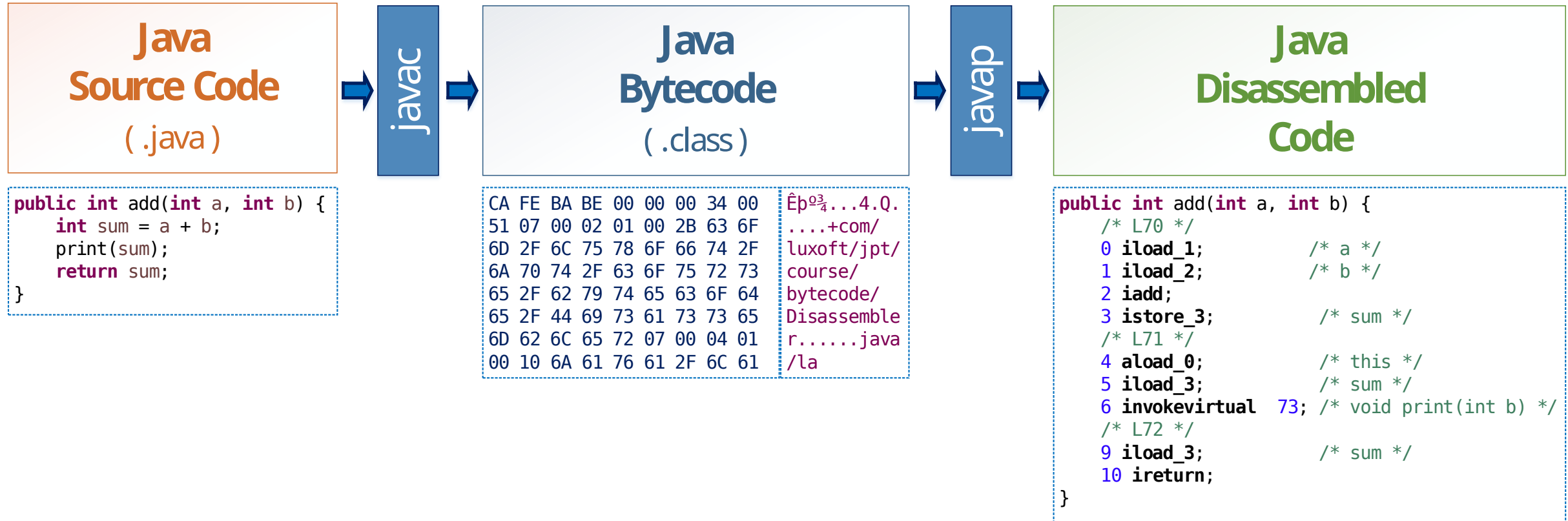
Eclipse: Bytecode Visualizer

IntelliJ IDEA: ASM Bytecode Outline



Understanding Bytecode

From Source Code to Disassembled Code



Understanding Bytecode

.class format

```
ClassFile {  
    u4          magic;  
    u2          minor_version;  
    u2          major_version;  
    u2          constant_pool_count;  
    cp_info     constant_pool[constant_pool_count-1];  
    u2          access_flags;  
    u2          this_class;  
    u2          super_class;  
    u2          interfaces_count;  
    u2          interfaces[interfaces_count];  
    u2          fields_count;  
    field_info   fields[fields_count];  
    u2          methods_count;  
    method_info  methods[methods_count];  
    u2          attributes_count;  
    attribute_info attributes[attributes_count];  
}
```

CA FE BA BE 00 00 00 34 00 51	Êþ ⁰³ / ₄ ...4.Q..
07 00 02 01 00 2B 63 6F 6D 2F	...+com/
6C 75 78 6F 66 74 2F 6A 70 74	luxoft/jpt/
2F 63 6F 75 72 73 65 2F 62 79	course/
74 65 63 6F 64 65 2F 44 69 73	bytecode/
61 73 73 65 6D 62 6C 65 72 07	Disassembler
00 04 01 00 10 6A 61 76 61 2F	java/
6C 61 6E 67 2F 4F 62 6A 65 63	lang/
74 01 00 06 3C 69 6E 69 74 3E	Object...<in
01 00 03 28 29 56 01 00 04 43	it>...()V...
6F 64 65 0A 00 03 00 09 0C 00	Code.....
05 00 06 01 00 0F 4C 69 6E 65	LineNumber
4E 75 6D 62 65 72 54 61 62 6C	berTable...L
65 01 00 12 4C 6F 63 61 6C 56	ocalVariable
61 72 69 61 62 6C 65 54 61 62	Table...this
6C 65 01 00 04 74 68 69 73 01	.

Understanding Bytecode

Runtime Constant Pool

- ♦ Ordered set of constants used by the type
- ♦ Entries have usually 2 bytes
- ♦ Plays a central role in the dynamic linking of Java programs

Runtime Constant Pool
#1: CONSTANT_Class
#2: CONSTANT_Fieldref
#3: CONSTANT_Methodref
#4: CONSTANT_InterfaceMethodref
#5: CONSTANT_String
#6: CONSTANT_Integer
#7: CONSTANT_Float
#8: CONSTANT_Long
#9: CONSTANT_Double
#10: CONSTANT_NameAndType
#11: CONSTANT_Utf8

Understanding Bytecode

Bytecode OpCode Families

```
public int add(int a, int b) {
    /* L70 */
    0 iload_1;          /* a */
    1 iload_2;          /* b */
    2 iadd;
    3 istore_3;         /* sum */
    /* L71 */
    4 aload_0;          /* this */
    5 iload_3;          /* sum */
    6 invokevirtual 73; /* void print(int b) */
    /* L72 */
    9 iload_3;          /* sum */
    10 ireturn;
}
```

ARITHMETIC OPCODES

Opcode	Description
add	add two values from top of stack and store the result
sub	subtracts two values from top of stack and store the result
div	divide two values from top of stack and store the result
mult	multiplies two values from top of stack and store the result

INVOCATION OPCODES

Opcode	Description
invokeinterface	calls an interface method
invokespecial	calls a non-overridable instance method
invokevirtual	calls a regular overridable instance method
invokestatic	calls a static method
invokedynamic	dynamic invocation

LOAD & STORE OPCODES

Opcod e	Description
load	load a value from Local Variable Array into the stack
ldc	load a constant from the Constant Pool into the stack
store	store a value into Local Variable Array and remove it from stack
dup	duplicate the value on top of the stack

Understanding Bytecode

Bytecode sample

```
public int add(int a, int b) {  
    int sum = a + b;  
    print(sum);  
    return sum;  
}
```



```
public int add(int a, int b) {  
    /* L70 */  
    0 iload_1;  
    1 iload_2;  
    2 iadd;  
    3 istore_3;  
    /* L71 */  
    4 aload_0;  
    5 iload_3;  
    6 invokevirtual 73;  
    /* L72 */  
    9 iload_3;  
    10 ireturn;  
}
```

Local Variable Array
index

/* a */
/* b */

/* sum */

/* this */
/* sum */
/* void print(int sum) */

/* sum */

Runtime Constant Pool
reference

Understanding Bytecode

How Operand Stack and Local Variable Array work

```
public int add(int a, int b) {  
    /* ... */  
    0 iload_1;          /* a=100 */  
    1 iload_2;          /* b=98 */  
    2 iadd;  
    3 istore_3;         /* sum */  
    /* ... */  
}
```

Stack Frame

Local Variable Array			
this	100	98	

Operand Stack

Local Variable Array			
this	100	98	

iload_1;
100
Operand Stack

Local Variable Array			
this	100	98	

iload_2;
98
100
Operand Stack

Local Variable Array			
this	100	98	

iadd;
198
Operand Stack

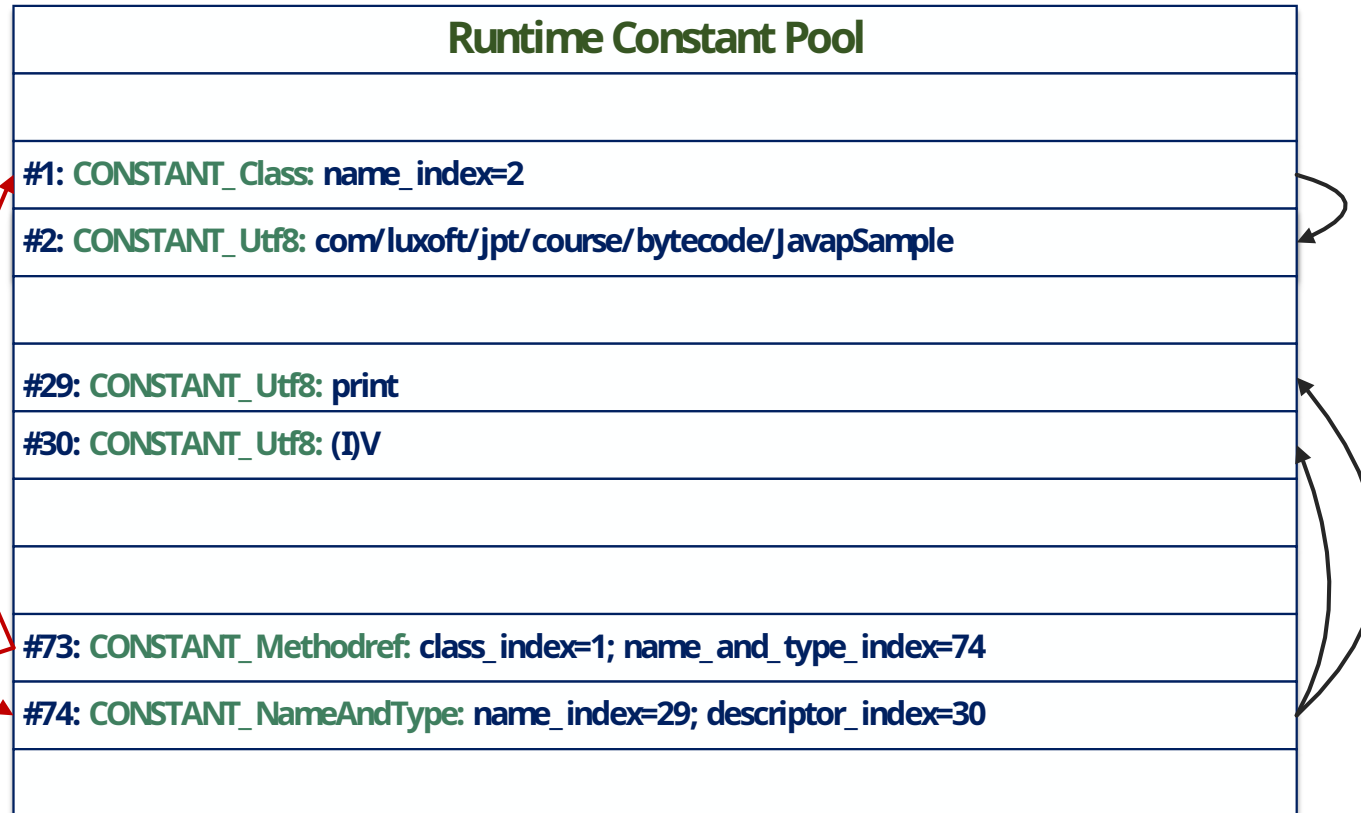
Local Variable Array			
this	100	98	198

istore_3;
Operand Stack

Understanding Bytecode

How Runtime Constant Pool works

```
public int add(int a, int b) {  
    /* ... */  
    6 invokevirtual 73;    /* void print(int sum) */  
    /* ... */  
}
```



Understanding Bytecode

How the Java Virtual Machine handles exceptions

```
public static int remainder(int dividend, int divisor) throws DivideByZeroException {  
    try {  
        return dividend % divisor;  
    } catch (ArithmeticException e) {  
        throw new DivideByZeroException();  
    }  
}
```

```
public static int remainder(int dividend, int divisor) throws com.luxoft.jpt.course.bytecode.DivideByZeroException {  
    /* L26 */  
    0 iload_0;          /* dividend */  
    1 iload_1;          /* divisor */  
    2 irem;  
    3 ireturn;  
    /* L27 */  
    4 astore_2;         /* e */  
    /* L28 */  
    5 new 17;  
    8 dup;  
    9 invokespecial 19;  /* com.luxoft.jpt.course.bytecode.DivideByZeroException() */  
    12 athrow;  
  
    /*  
        ExceptionTable  
    */  
    /* -----+-----+-----+----- */  
    /* start_pc | end_pc | handler_pc | catch_type */  
    /* -----+-----+-----+----- */  
    /*      0 |      3 |      4 | java.lang.ArithmeticException */  
    /* -----+-----+-----+----- */  
}
```

Understanding Bytecode

Case Study

javap - Java Class File Disassembler

```
javap -c -verbose ./com/luxoft/jpt/course/bytecode/JavapSample

Classfile /C:/java-performance-and-tuning/target/classes/com/luxoft/jpt/course/bytecode/JavapSample.class
  minor version: 0
  major version: 52
...
Constant pool:
  #1 = Class           #2          // com/luxoft/jpt/course/bytecode/JavapSample
  #2 = Utf8            com/luxoft/jpt/course/bytecode/JavapSample
  #3 = Class           #4          // java/lang/Object
...
{
  public static int remainder(int, int) throws com.luxoft.jpt.course.bytecode.DivideByZeroException;

  Exceptions:
    throws com.luxoft.jpt.course.bytecode.DivideByZeroException

    0: iload_0
    1: iload_1
    2: irem
...
  Exception table:
    from   to  target type
      0     3     4   Class java/lang/ArithmeticException
}
```

Bytecode Execution Workflow

Execution Engine

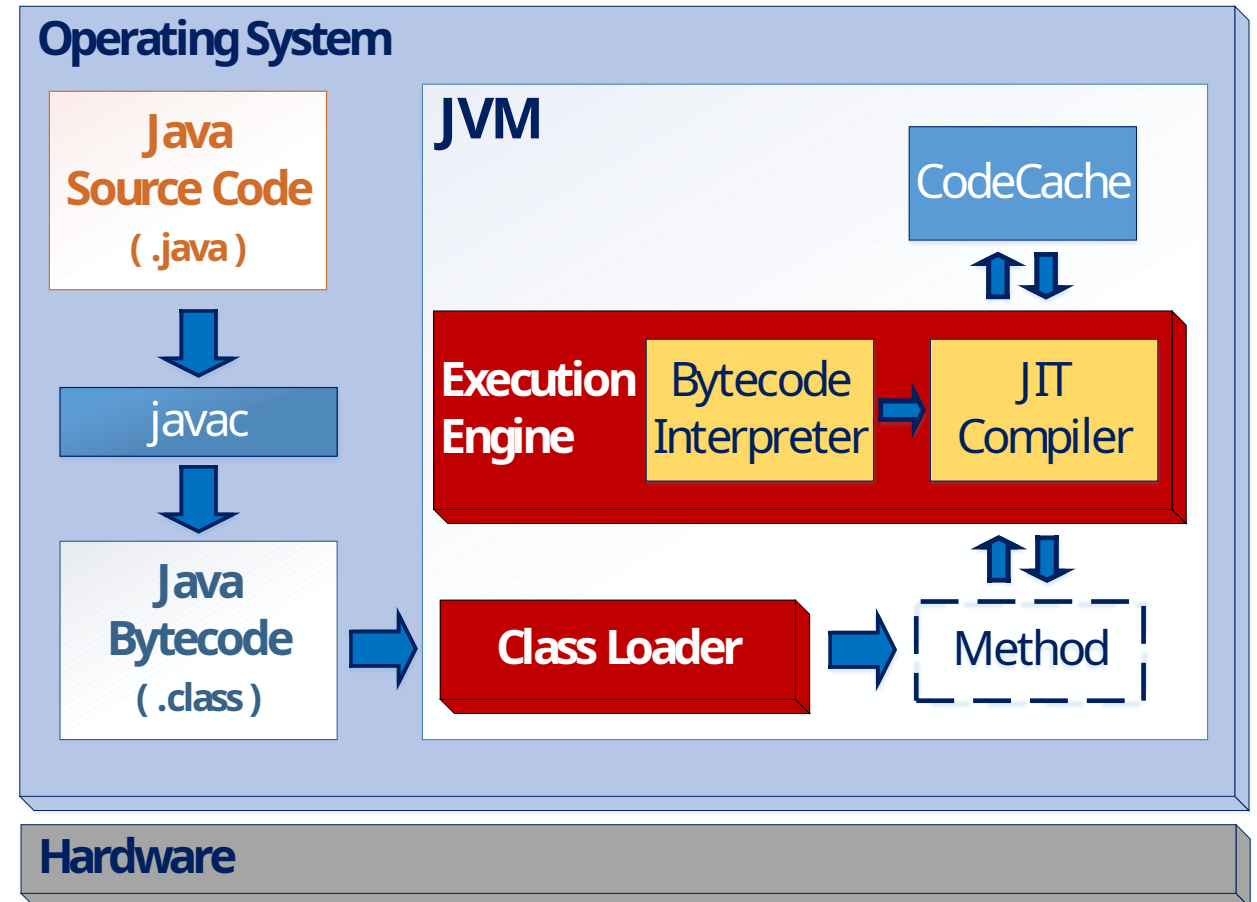
- ♦ **HotSpot** rationales

1. Weak Generation Hypothesis
 - ♦ memory management and Garbage Collector
2. use runtime information to take best decisions and to continuously optimize
 - ♦ **Execution Engine**

- ♦ **Execution Engine**

Interpreter

Just-In-Time Compiler



Bytecode Execution Workflow

HotSpot Interpreter

- ◆ When the JVM first starts up, thousands of methods are called

Issue: compiling all of these methods can significantly affect startup time

Solution: methods are not compiled the first time they are called, they are **interpreted**

 - ◆ HotSpot JVM starts running in **Interpreter** mode
- ◆ Each method have a call count, once it reaches an **invocation threshold** it gets compiled
hence **Hot Spots** (HotSpot JVM)

JVM argument	Explanations
-XX:CompileThreshold	Number of method invocations/branches before compiling

Bytecode Execution Workflow

HotSpot Just-In-Time Compiler

♦ JIT Compiler shapes

Client (-client) C1

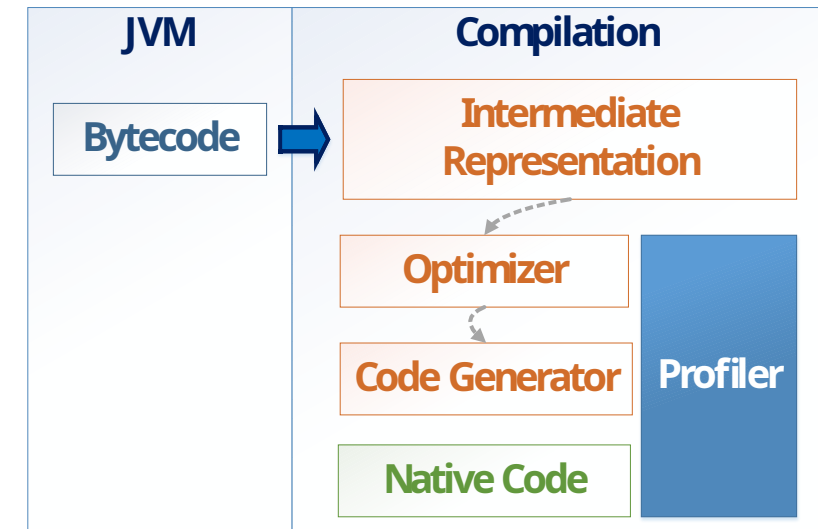
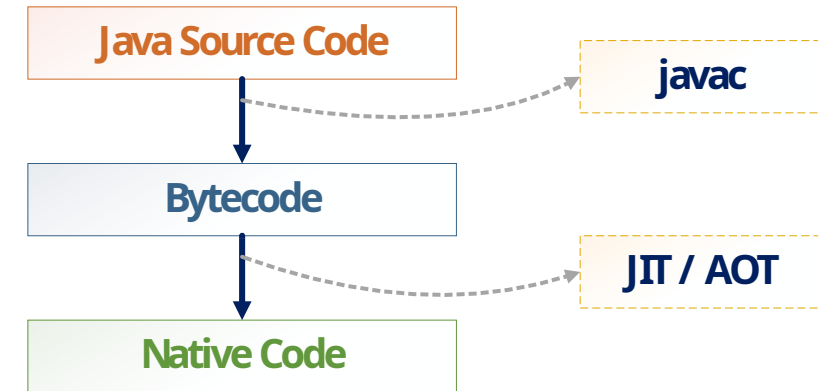
- ♦ fast code generation of acceptable quality
- ♦ rock-solid and proved optimizations
- ♦ doesn't need a **Profiler**
- ♦ compilation threshold: **1,5k** invocations

Server (-server) C2

- ♦ highly optimized code
- ♦ many aggressive and speculative optimizations
- ♦ does need a **Profiler**
- ♦ compilation threshold: **10k** invocations

Tiered Mode (-XX:+TieredCompilation) C1 + C2

- ♦ Level 0 = Interpreter
- ♦ Level 1-3 = **C1**
 - ♦ **C1** without profiling
 - ♦ **C1** with basic profiling
 - ♦ **C1** with full profiling
- ♦ Level 4 = **C2**



Bytecode Execution Workflow

HotSpot Just-In-Time Compiler

♦ JIT Compiler optimizations

dynamic de-virtualization

- ♦ if JIT only sees 1 implementation of a virtualized method
it becomes a mono-morphic call

dynamic de-optimization

- ♦ if JIT discovers another implementation it must *de-optimize* and might *re-optimize* for second implementation
it becomes a bi-morphic call

Intrinsics

- ♦ optimizations done specifically to CPU
- ♦ do not inline, but inserts “best” native code
- ♦ Examples: `CAS`, `String::equals`; `Math::*`; `System::arrayCopy`; `Object::hashCode`

Method inlining

```
double getInterestRate() {  
    return (canComputeInterestRate()) ? 0 : (3 * amount / 1200);  
}  
  
boolean canComputeInterestRate() {  
    return amount != 0;  
}
```



```
double getInterestRate_() {  
    return (amount != 0) ? 0 : (3 * amount / 1200);  
}
```

Bytecode Execution Workflow

HotSpot Just-In-Time Compiler

Loop unrolling – reduces cost of jump instructions that control the loop

```
for (x = 0; x < 100; x++) {  
    compute(x);  
}
```



```
for (x = 0; x < 100; x += 5) {  
    compute(x);  
    compute(x + 1);  
    compute(x + 2);  
    compute(x + 3);  
    compute(x + 4);  
}
```

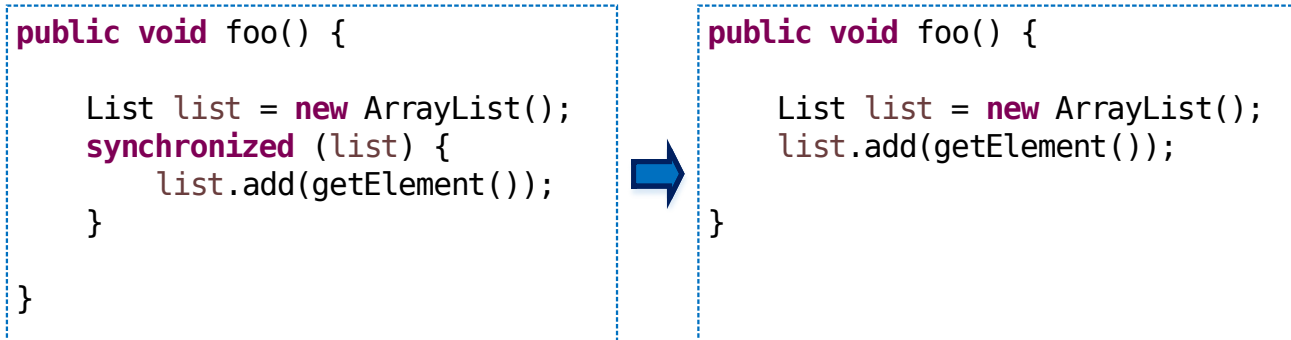
Null check elimination – eliminates unreachable null code branch

```
private void enroll() {  
    JavaPerformanceAndTuning jtp = new JavaPerformanceAndTuning();  
    if (null == jtp)  
        throw new NullPointerException("Cannot enroll to null course");  
    jtp.enroll();  
}
```

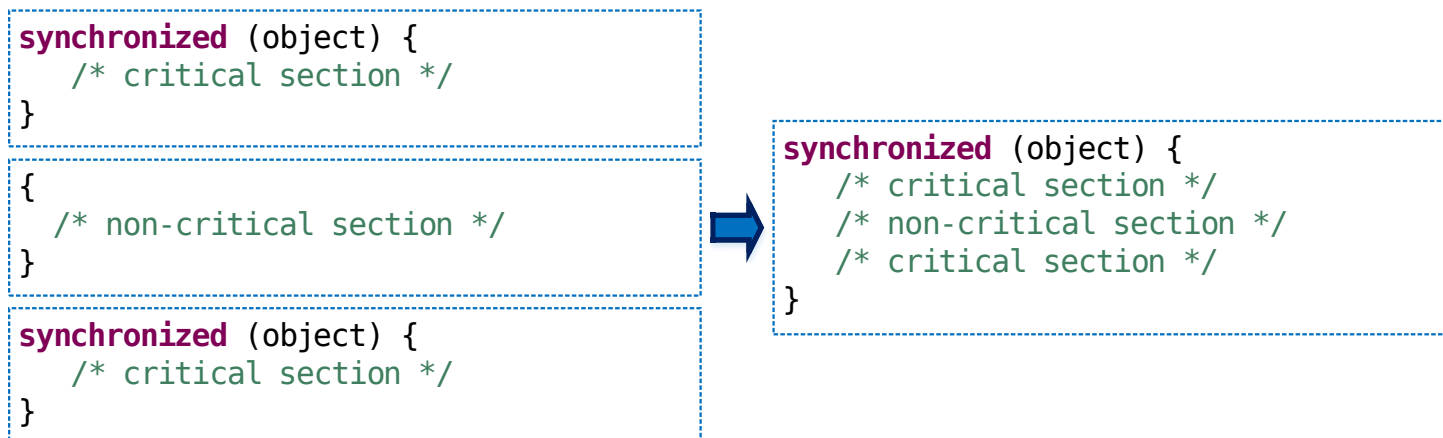
Bytecode Execution Workflow

HotSpot Just-In-Time Compiler

Lock Eliding – when only the thread that created the lock will ever have access to it, can safely be omitted



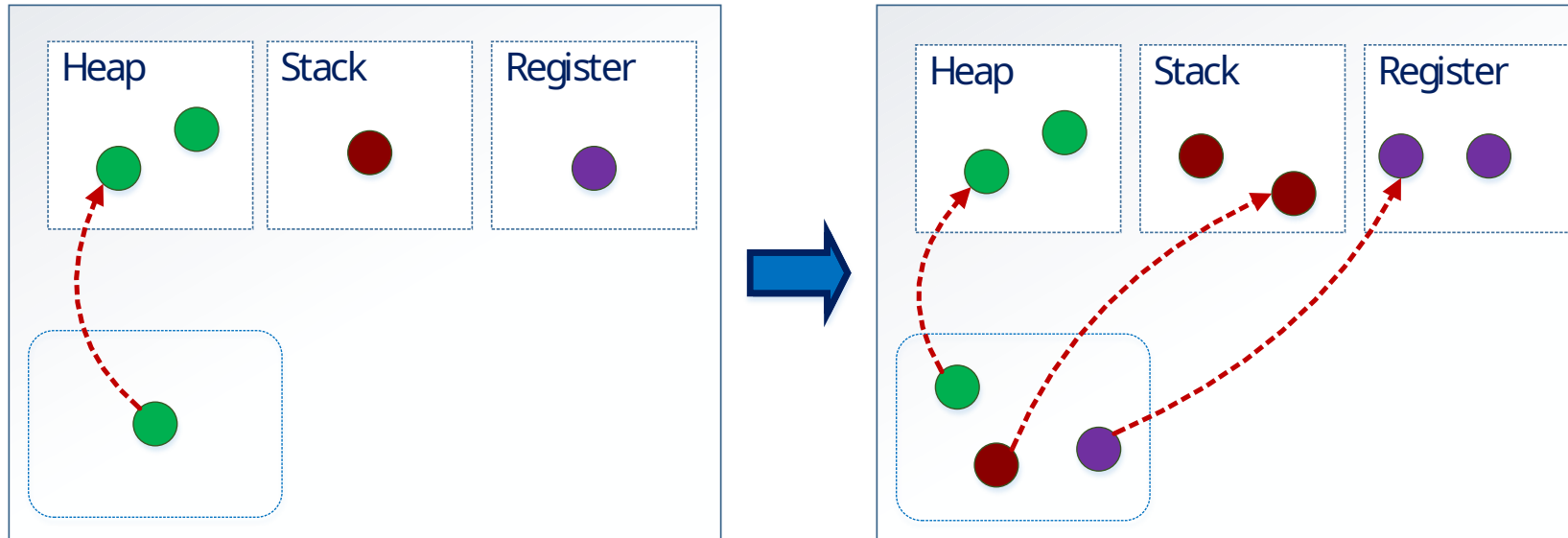
Lock coarsening – merges two critical sections, along with the non-critical section, into a larger block



Bytecode Execution Workflow

HotSpot Just-In-Time Compiler

- ♦ **Escape Analysis** - Java Hotspot Compiler can analyze the scope of a new object's uses and decide whether to allocate it on the Java heap



JVM arguments	Explanations
<code>-XX:+DoEscapeAnalysis</code>	Allocation optimization. Enabled by default in Java 6 u23

Bytecode Execution Workflow

HotSpot JVM arguments

JVM argument	Explanations
<code>-Djava.compiler=none</code>	Disables JIT Compiler
<code>-client</code>	Activates client (C1) compilation
<code>-server</code>	Activates server (C2) compilation
<code>-XX:+TieredCompilation</code>	Activates tiered (C1+C2) compilation
<code>-XX:+PrintCompilation</code>	Prints time spent in JIT Compiler
<code>-XX:+PrintInlining</code>	Prints what methods get inlined
<code>-XX:-CITime</code>	Prints time spent in JIT Compiler.
<code>-XX:MaxInlineSize</code>	Specifies maximum number of bytecode instructions in a method which gets inlined. It does not look at the invocation count
<code>-XX:FreqInlineSize</code>	Specifies maximum number of bytecode instructions in a frequently executed method which gets inlined. It looks at the invocation count

How to see Native Code from Bytecode

Assembly Code

- ◆ How to print out Java compiled assembly instructions ?

Build **HSDIS** library

- ◆ <https://github.com/AdoptOpenJDK/jitwatch/wiki/Building-hsdis>

Copy **HSDIS** library under

- ◆ `%JAVA_HOME% / jre / ...`

JVM argument	Description
<code>-XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly</code>	Activates for assembly printing

From Bytecode to Native Code in HotSpot JVM

Case Study

- ❑ Direct vs. Getter/Setter fields access members
 - ❑ interpreted mode
 - ❑ interpreted + compiled mode
- ❑ Mono-morphic / bi-morphic / poli-morphic virtual method inlining
- ❑ JIT Watch <https://github.com/AdoptOpenJDK/jitwatch>

THANK YOU

Appendix

From Bytecode to Native Code in HotSpot JVM

Read PrintCompilation

♦ 202 15 % 3 ! s b n com.luxoft.jpt.course.jit.fieldaccess.DfaCaller::call @ 16 (51 bytes)

- ♦ 202 : millis from start JVM
- ♦ 15: sequence number of compilation
- ♦ %: On Stack Replacement
- ♦ 3: Compilation Level
- ♦ !: method has exception handlers
- ♦ s: synchronized method
- ♦ b: compilation occurred in blocking mode
- ♦ n: compilation occurred for a wrapper to a native method
- ♦ 16: On Stack Replacement bytecode index {osr_bci}
- ♦ 51 bytes: number of bytes contained in compiled method

Bytecode Execution Workflow

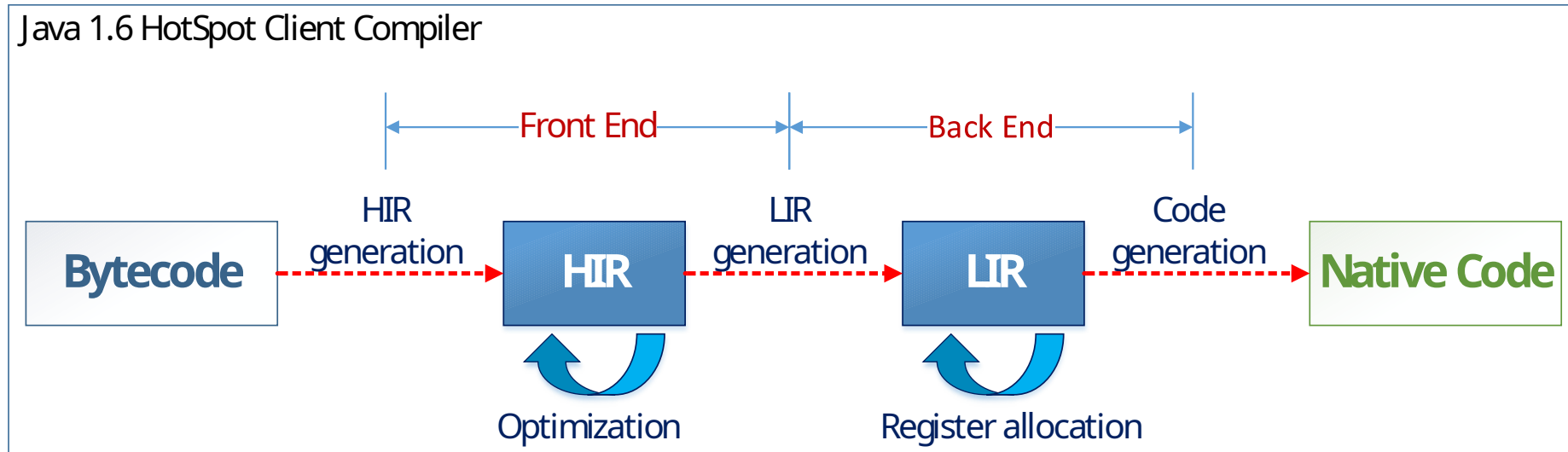
HotSpot Just-In-Time Compiler

- ♦ **Code Cache** – holds JVM compiled code (set of assembly-language instructions)
“hidden space” cleaned by JIT Compiler threads
- ♦ **Warnings:**
 - ♦ CodeCache is full. Compiler has been disabled
 - ♦ CodeCache is full. Try increasing the code cache size using `-XX:ReservedCodeCacheSize`

JVM arguments	Explanations
<code>-XX:InitialCodeCacheSize</code>	Initial code cache size (in bytes)
<code>-XX:ReservedCodeCacheSize</code>	Maximum/reserved code cache size to be used
<code>-XX:+UseCodeCacheFlushing</code>	Attempt to sweep the code cache before shutting off compiler. Default with Java 7u4

Bytecode Execution Workflow

HotSpot Just-In-Time Compiler



- ♦ **SSA** – Static Single Assignment
- ♦ **CFG** – Control Flow Graph
- ♦ **HIR** – High Level Intermediate Representation
- ♦ **LIR** – Low Level Intermediate Representation

- ♦ **HIR** optimizations:
 - method inlining
 - loop unrolling
 - null check elimination
- ♦ **LIR** register allocation:
 - maps virtual to physical registers